



IES GREGORI MAÏANS
OLIVA

/ Departamento de Tecnología

SA URBANLAB 1.0: DISEÑA LA CIUDAD QUE QUIERES

BLOQUE III: DOSSIER DEL ALUMNO

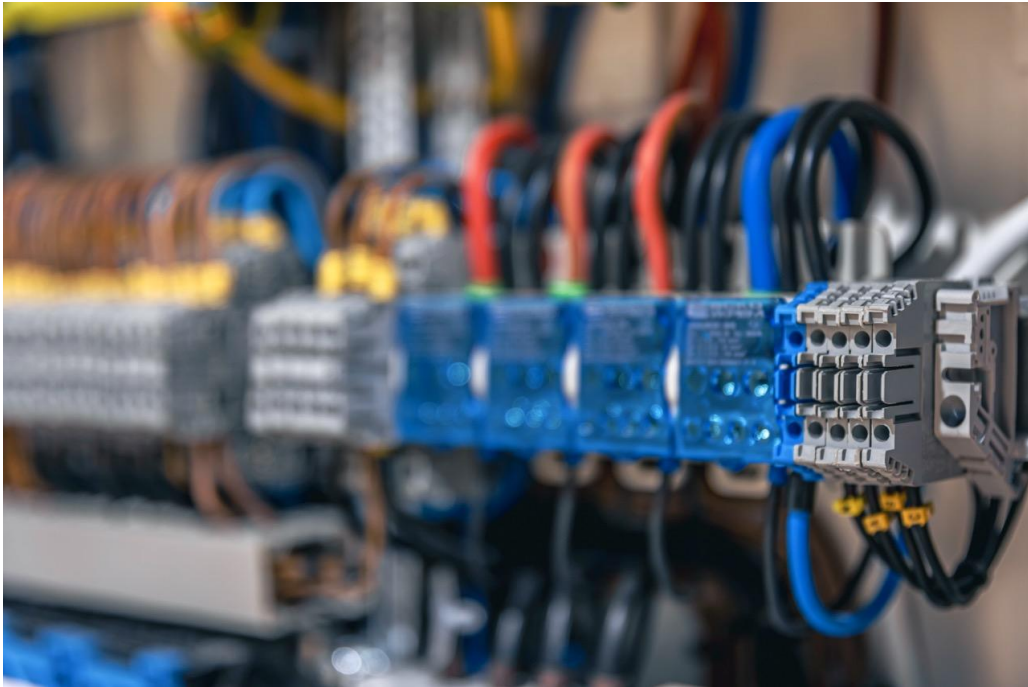


MATERIA: Tecnología e Ingeniería I

CURSO: 1º de Bachillerato

- 1. Sistemas eléctricos**
- 2. Sistemas informáticos**
- 3. Automatización**

1. SISTEMAS ELÉCTRICOS



Resumen Ejecutivo

Este documento sintetiza los principios fundamentales, componentes y leyes que rigen los circuitos eléctricos de corriente continua (CC). La electricidad se define como el movimiento de electrones a través de un conductor, fluyendo convencionalmente del polo positivo al negativo. El estudio de estos sistemas se basa en tres magnitudes críticas: **intensidad (Amperios)**, **resistencia (Ohmios)** y **voltaje (Voltios)**, las cuales están interrelacionadas por la **Ley de Ohm**.

Los circuitos pueden configurarse de diversas formas (serie, paralelo o mixtos), cada una con comportamientos específicos de tensión y corriente. Además de los componentes pasivos, se analizan elementos de maniobra como relés, dispositivos de protección como fusibles e interruptores diferenciales, y componentes semiconductores como diodos y transistores. Finalmente, para el análisis de redes complejas, se establecen las **Leyes de Kirchhoff** como la herramienta técnica fundamental para determinar variables en nodos y mallas.

1. Fundamentos de la Electricidad

La materia está compuesta por átomos con núcleos de protones (positivos) y neutrones, rodeados por electrones (negativos). La corriente eléctrica es el flujo de electrones que pasan de un átomo a otro a través de un material conductor.

Sentido de la Corriente

- **Sentido Real:** Los electrones fluyen desde un átomo con exceso de carga negativa hacia uno con falta de carga (positivo).
- **Sentido Convencional:** Utilizado en el diseño de circuitos, establece el flujo del polo positivo (+) al negativo (-).

Conductividad de los Materiales

La capacidad de flujo depende del material. Metales como el oro, la plata, el cobre y el aluminio se clasifican como excelentes conductores. La resistencia específica de un material se denomina **resistividad ρ** , la cual varía con la temperatura (aumenta en metales y decrece en semiconductores).

Material	Resistividad ($\Omega \cdot m$ a 23°C)
Plata	$1,59 \cdot 10^{-8}$
Cobre	$1,68 \cdot 10^{-8}$
Oro	$2,20 \cdot 10^{-8}$

Aluminio	$2,65 \cdot 10^{-8}$
Hierro	$9,71 \cdot 10^{-8}$
Vidrio	10^{12}
Caucho	$75 \cdot 10^{18}$

2. Magnitudes Eléctricas y Leyes Fundamentales

El comportamiento de un circuito se define mediante las siguientes magnitudes y sus relaciones matemáticas:

Magnitudes Principales

- **Intensidad (I):** Cantidad de carga que atraviesa un conductor por unidad de tiempo. Se mide en **Amperios (A)**. El aparato de medida es el amperímetro (conectado en serie).

- Fórmula: $I = Q / t$

- **Resistencia (R):** Oposición al paso de la corriente. Se mide en **Ohmios (Ω)**. El ohmímetro se coloca en los bornes del componente.

- Fórmula: $R = \rho \cdot (L/s)$ (donde L es longitud y s es sección).

- **Voltaje o Tensión (V/U):** Fuerza que empuja a los electrones entre dos puntos. Se mide en **Voltios (V)** mediante un voltímetro (conectado en paralelo).

- **Potencia (P):** Velocidad de uso o transporte de energía. Se mide en **Vatios (W)**.

- Fórmula: $P = V \cdot I$

- **Energía (E):** Trabajo obtenido de la corriente. Se mide en **Julios (J)** o Kilovatios-hora (KWh).

- Fórmula: $E = P \cdot t$

Ley de Ohm y Efecto Joule

- **Ley de Ohm:** Establece que el voltaje es igual al producto de la resistencia por la intensidad ($V = R \cdot I$).

- **Efecto Joule:** Describe la pérdida de energía en forma de calor cuando la corriente atraviesa un conductor: $E_{\text{perdida}} = R \cdot I^2 \cdot t$.

3. Tipos de Corriente y Generadores

Existen dos categorías principales de corriente eléctrica:

1. **Corriente Continua (CC/DC):** Flujo constante en un solo sentido y con valor fijo. Producida por pilas, baterías, dinamos y placas fotovoltaicas.
2. **Corriente Alterna (CA/AC):** Los electrones cambian de sentido periódicamente. En Europa, la red doméstica opera a 220V y 50Hz. Es generada por alternadores.

Acoplamiento de Generadores

- **En Serie:** La tensión total es la suma de las tensiones individuales ($V = V1 + V2...$). La intensidad es constante.
- **En Paralelo:** La tensión se mantiene igual a la de un solo generador. La intensidad total es la suma de las intensidades de cada rama. Todos los generadores deben ser idénticos.

4. Componentes del Circuito

Elementos de Control y Maniobra

- **Interruptor:** Abre o cierra el circuito de forma permanente.
- **Pulsador:** Actúa solo mientras se presiona (pueden ser Normalmente Abiertos - NA o Normalmente Cerrados - NC).
- **Conmutador:** Permite dirigir la corriente hacia dos circuitos distintos o controlar un receptor desde dos puntos.
- **Relé:** Dispositivo donde una corriente pequeña activa un electroimán que mueve conmutadores para controlar circuitos de mayor potencia.

Elementos de Protección

- **Fusible:** Hilo que se funde ante una intensidad excesiva, abriendo el circuito.
- **Interruptor Magnetotérmico:** Se abre automáticamente ante cortocircuitos o sobrecargas.
- **Diferencial:** Protege a las personas detectando fugas de corriente a tierra. Se dispara si la diferencia entre la corriente de entrada y salida supera la sensibilidad (30 mA en uso doméstico).

Receptores y Componentes Electrónicos

- **Condensadores:** Almacenan carga eléctrica. Su capacidad (C) se mide en Faradios (F). Poseen una constante de tiempo $\tau = R \cdot C$.

- **Diodos y LEDs:** Permiten el paso de corriente en un solo sentido (polarización directa). Los LEDs convierten la energía en luz sin calentarse.

- **Transistores:** Actúan como amplificadores o interruptores controlados por la corriente de base. Operan en tres zonas: corte (no conduce), activa (conducción proporcional) y saturación (conducción máxima).

- **Resistencias Variables:**

- **Potenciómetro:** Ajuste manual.
- **LDR:** Varía según la luz (más luz, menos resistencia).
- **Termistor (NTC):** Resistencia que disminuye al aumentar la temperatura.

5. Análisis de Asociaciones y Leyes de Kirchhoff

Configuración de Receptores

- **Serie:** Los receptores se conectan uno tras otro. La resistencia total es la suma de las individuales. Si uno falla, el circuito se interrumpe.

- **Paralelo:** Cada receptor se conecta directamente a los polos del generador. La tensión es la misma para todos. Si uno falla, el resto sigue funcionando.

- **Mixta:** Combinación de ambas, cuya resolución requiere simplificar el circuito por etapas.

Leyes de Kirchhoff

Para circuitos complejos que no se resuelven solo con la Ley de Ohm, se aplican estas dos reglas:

1. **Primera Ley (Nodos):** La suma de las intensidades que entran a un nodo es igual a la suma de las que salen $(\sum I_{entran} = \sum I_{salen})$.

2. **Segunda Ley (Mallas):** En una trayectoria cerrada, la suma de las fuerzas electromotrices es igual a la suma de las caídas de tensión en las resistencias $(\sum E = \sum R \cdot I)$.



1. Lenguajes de programación textual. Características, elementos y lenguajes.

Un lenguaje de programación textual es un lenguaje formal que permite a los programadores escribir instrucciones y algoritmos mediante líneas de código de texto. Requiere un alto grado de abstracción y aprendizaje de una gramática específica para controlar el comportamiento de sistemas informáticos.

Características Principales

- **Sintaxis estricta:** Reglas de escritura obligatorias; un solo error de puntuación genera fallos.
- **Abstracción:** Mayor control sobre el hardware o la lógica de software que en la programación visual.
- **Versatilidad:** Permiten desarrollar desde inteligencia artificial hasta sistemas embebidos.

Elementos Fundamentales

- **Palabras reservadas:** Términos propios del lenguaje con un significado especial (ej. *if*, *for*).
- **Variables y Constantes:** Espacios de memoria para almacenar datos que pueden cambiar o mantenerse fijos.
- **Tipos de datos:** Definen la naturaleza de la información (texto, números enteros, decimales, booleanos).
- **Operadores:** Símbolos para realizar operaciones matemáticas ($\backslash(+\backslash)$, $\backslash(-\backslash)$, $\backslash(*\backslash)$) o lógicas.
- **Estructuras de control:** Bloques lógicos como bucles (*loops*) y condicionales (*if/else*) que dirigen el flujo del programa.

Lenguajes Más Populares

- **Python:** Muy legible, similar al inglés. Destaca en IA y análisis de datos.
- **JavaScript:** Imprescindible para el desarrollo web e interactividad en navegadores.
- **Java:** Orientado a objetos, muy utilizado en aplicaciones empresariales y Android.
- **C++ / C#:** Lenguajes de alto rendimiento, usados en videojuegos y software de escritorio.

2. Proceso de desarrollo: edición, compilación o interpretación, ejecución, pruebas y depuración.

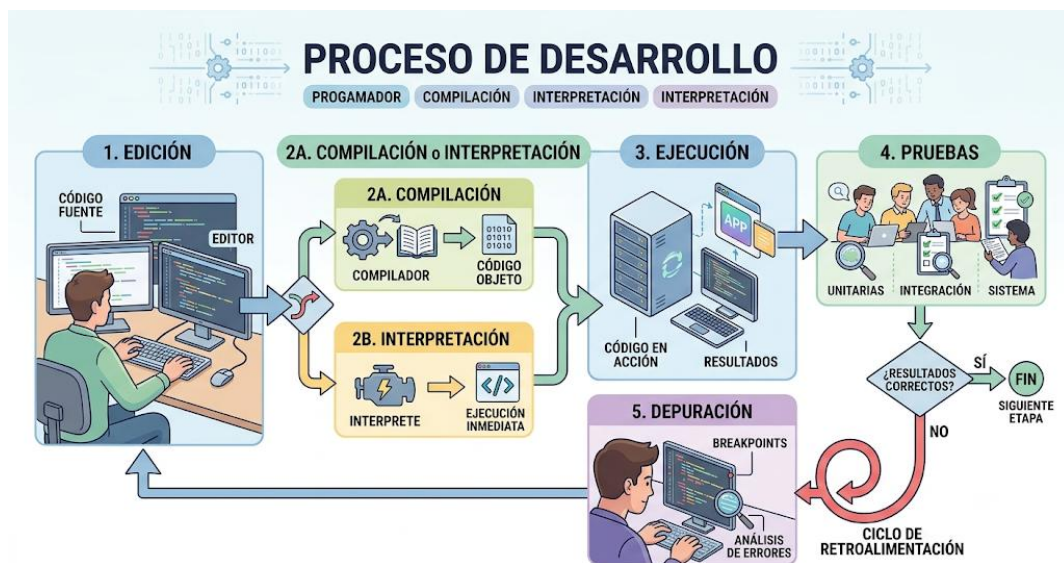
El proceso de desarrollo de software es el ciclo estructurado que transforma una idea lógica en un programa funcional y sin errores.

El Ciclo de Desarrollo

- **Edición:** El programador escribe el código fuente en texto plano utilizando un editor de texto o Entorno de Desarrollo Integrado (IDE).
- **Compilación o Interpretación:** Fase de traducción del código fuente a código máquina comprensible por el procesador.
- **Ejecución:** El sistema operativo carga el programa traducido en la memoria y el procesador procesa las instrucciones.
- **Pruebas:** Fase de verificación donde se ejecuta el programa bajo distintos escenarios para comprobar que cumple los requisitos.
- **Depuración:** Proceso de identificar, rastrear y corregir los errores (*bugs*) descubiertos durante las pruebas o la ejecución.

Compilación vs. Interpretación

- **Compilación:** Traduce todo el código de golpe antes de ejecutarlo, generando un archivo binario independiente (ej. en C++ o Rust).
- **Interpretación:** Traduce y ejecuta el código línea por línea en tiempo real mediante un programa llamado intérprete (ej. en Python o JavaScript).





3. Creación de programas para la resolución de problemas. Modularización.

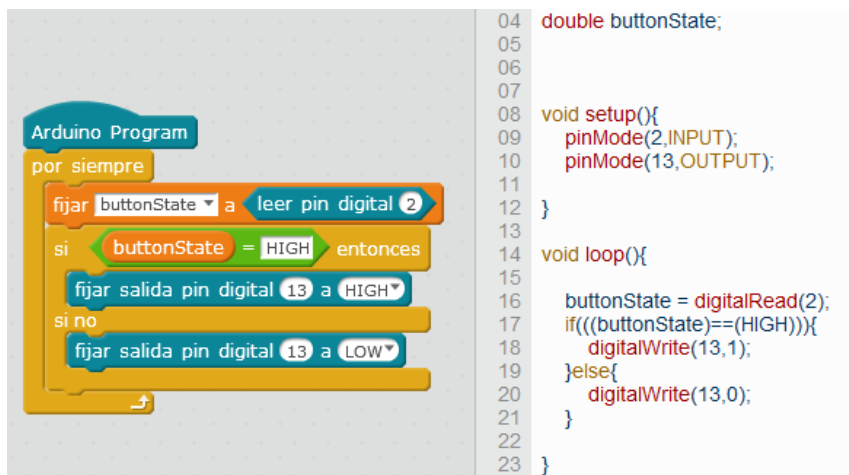
La creación de programas para la resolución de problemas es un enfoque metodológico que transforma una necesidad o conflicto del mundo real en una solución informática automatizada. Este proceso no empieza escribiendo código, sino entendiendo a fondo el problema.

Fases de la Resolución de Problemas con Software

- **Análisis del problema:** Identificar claramente qué datos se tienen (entradas), qué resultado se busca (salidas) y qué restricciones existen.
- **Diseño del algoritmo:** Crear la secuencia lógica de pasos ordenados y finitos para resolver el problema, sin usar un lenguaje específico.
- **Codificación:** Traducir el algoritmo diseñado a un lenguaje de programación textual (como Python, Java o C++).
- **Validación:** Comprobar con datos de prueba reales y extremos que el programa ofrece la solución correcta en todos los casos.

Herramientas para Diseñar la Lógica

- **Pseudocódigo:** Narración de los pasos del programa en un lenguaje humano estructurado similar a la programación.
- **Diagramas de flujo:** Representación gráfica visual del flujo de datos y las decisiones lógicas del algoritmo.



Para ilustrar el proceso completo, resolveremos un problema común en el ámbito escolar o empresarial: **Calcular el promedio de notas de un alumno y determinar si ha aprobado.**



1. Análisis del Problema

- **Entradas:** Tres notas numéricas (Nota 1, Nota 2, Nota 3).
- **Restricciones:** Las notas deben ser números entre 0 y 10. La nota mínima para aprobar es 5.
- **Salidas:** El promedio final y un mensaje que indique "Aprobado" o "Reprobado".

2. Diseño del Algoritmo (Pseudocódigo)

Este es el mapa lógico antes de tocar cualquier línea de código real:

Algoritmo CalcularPromedio

 Escribir "Introduce la primera nota:"

 Leer nota1

 Escribir "Introduce la segunda nota:"

 Leer nota2

 Escribir "Introduce la tercera nota:"

 Leer nota3

$\text{promedio} = (\text{nota1} + \text{nota2} + \text{nota3}) / 3$

 Escribir "El promedio final es: ", promedio

 Si promedio ≥ 5 Entonces

 Escribir "Estado: Aprobado"

 Sino

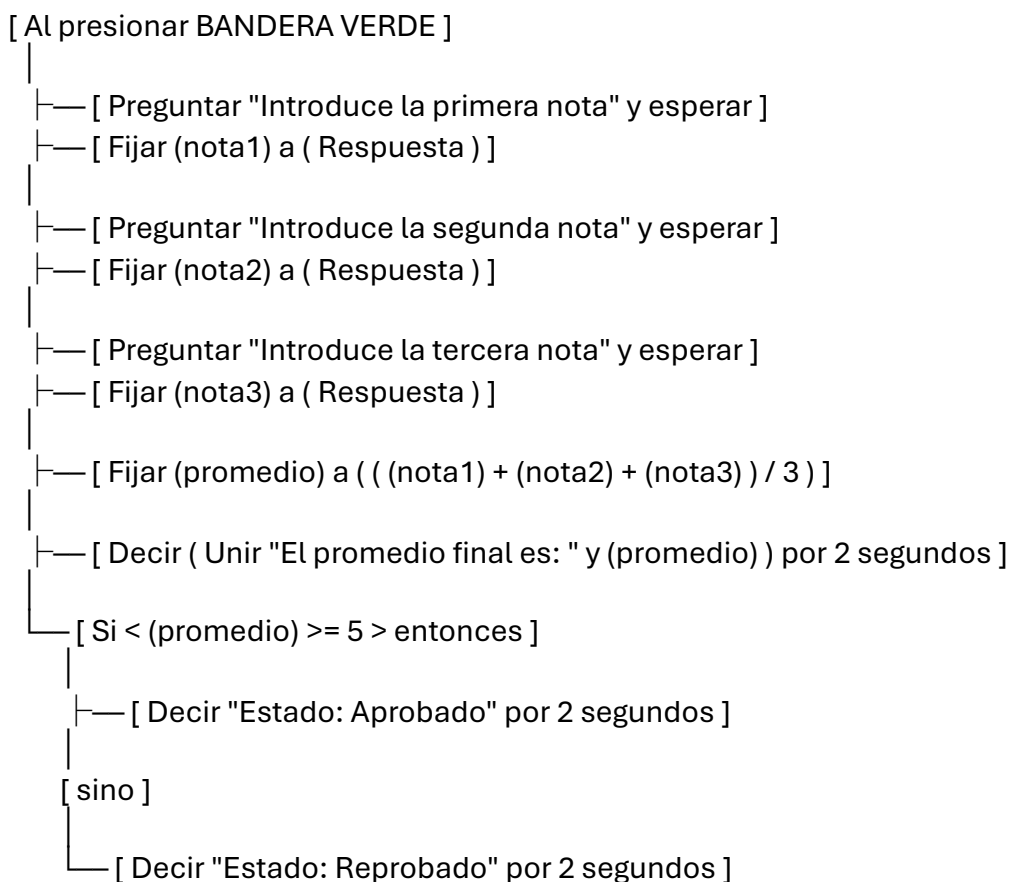
 Escribir "Estado: Reprobado"

 FinSi

FinAlgoritmo

3. Codificación

Traducimos el diseño anterior a **programación por bloques** (el estilo visual utilizado en plataformas como Tinkercad o App Inventor).



Esta es la guía exacta de colores por categorías para programar la lógica de bloques en Tinkercad Circuitos:

● Azul (Salida / Output)

- **Función:** Envía órdenes a los componentes del exterior (motores, luces, pantallas).
- **Bloques clave:** Definir pasador (para encender/apagar pines), Girar servo, Imprimir en monitor serie (para mostrar texto en la pantalla de depuración). [1, 2]

● Verde (Entrada / Input)

- **Función:** Lee la información del entorno o de los sensores.
- **Bloques clave:** Leer pasador digital, Leer pasador analógico, o leer componentes específicos como sensores de distancia o temperatura. [1]

● Morado (Notación / Comentarios)

- **Función:** No ejecuta ninguna acción lógica, sirve exclusivamente para organizar el proyecto.
- **Bloques clave:** Bloques de Comentario de texto plano para documentar qué hace cada sección del algoritmo. [1]

● Naranja (Control)

- **Función:** Governa el flujo del programa, los tiempos y las decisiones del algoritmo.

- **Bloques clave:** Bloques de espera (Esperar X segundos), bucles de repetición (Repetir X veces), y las estructuras condicionales esenciales para resolver problemas: Si... entonces y Si... entonces... si no. [1, 2]

● Verde Claro (Matemáticas / Math)

- **Función:** Realiza cálculos numéricos y comparaciones lógicas.

- **Bloques clave:** Operaciones aritméticas elementales ($\backslash(+)$, $\backslash(-)$, $\backslash(*)$, $\backslash(/)$) y los bloques hexagonales de comparación lógica ($\backslash(<)$, $\backslash(>)$, $\backslash(=)$, $\backslash(\geq)$, $\backslash(\leq)$). También aloja las condiciones compuestas (Y / O). [1]

■ Rosa / Magenta (Variables)

- **Función:** Almacena valores en la memoria que cambiarán a lo largo de la ejecución.

- **Bloques clave:** Crear variable (donde definirías nota1, nota2, nota3 o promedio), e instrucciones como Definir variable en... o Cambiar variable por....

¿Cómo quedaría nuestro programa de promedio de notas en este entorno?

Si arrastraras estos bloques en Tinkercad Circuitos, la lógica matemática y de control se vería estructurada de la siguiente manera:

1. ■ **Rosa (Variables):** Usas Definir [nota1] en [valor] para cargar los datos de las tres notas.
 2. ■ **Rosa + ● Verde Claro (Matemáticas):** Usas Definir [promedio] en y encajas dentro bloques verdes de suma y división para calcular $(\text{nota1} + \text{nota2} + \text{nota3}) / 3$.
 3. ● **Azul (Salida):** Usas Imprimir en monitor serie [promedio] para ver el número resultante en la consola.
 4. ● **Naranja (Control):** Arrastras un bloque condicional Si / Si no.
 5. ● **Verde Claro (Matemáticas):** Dentro de la condición del Si, encajas el bloque hexagonal de comparación: $[\text{promedio}] \geq [5]$. [1]
 6. ● **Azul (Salida):**
 - En la primera boca del bloque pones: Imprimir en monitor serie "Aprobado".
 - En la boca del *Si no* pones: Imprimir en monitor serie "Reprobado".
-

4. Validación y Pruebas

Probamos el programa con diferentes escenarios para asegurar su correcto funcionamiento:

- **Caso de prueba 1 (Aprobado):** Notas 6, 7, 8. Promedio = 7.0. Salida esperada: *Aprobado*.
- **Caso de prueba 2 (Reprobado):** Notas 3, 4, 5. Promedio = 4.0. Salida esperada: *Reprobado*.
- **Caso de prueba 3 (Límite):** Notas 5, 5, 5. Promedio = 5.0. Salida esperada: *Aprobado*.

La **modularización** es una técnica de diseño de software que consiste en dividir un programa grande y complejo en partes más pequeñas, independientes y manejables llamadas **módulos** o **funciones**.

Su principio fundamental es "*divide y vencerás*", permitiendo que cada módulo se encargue de resolver una sola tarea específica.

Características de la Modularización

- **Alta cohesión:** Cada módulo debe realizar una única tarea y hacerla bien (ej. un módulo solo calcula promedios, otro solo maneja la base de datos).
- **Bajo acoplamiento:** Los módulos deben ser lo más independientes posible entre sí. Cambiar el código dentro de un módulo no debería romper el funcionamiento de los demás.
- **Encapsulamiento:** Los detalles internos de cómo funciona un módulo quedan ocultos para el resto del programa; solo importa lo que recibe (entradas) y lo que devuelve (salidas).

Ventajas de Trabajar con Módulos

- **Reutilización de código:** Un módulo escrito una vez puede usarse en diferentes partes del programa (o en proyectos futuros) sin tener que volver a escribirlo.
- **Facilidad de mantenimiento:** Si se detecta un error (*bug*), es mucho más fácil localizarlo y corregirlo dentro de un módulo aislado que en un archivo de miles de líneas.

- **Trabajo en equipo:** Permite que varios programadores trabajen simultáneamente en el mismo proyecto, encargándose cada uno de módulos diferentes sin pisarse el trabajo.

- **Legibilidad:** El código principal se vuelve mucho más limpio y fácil de entender a primera vista.

Ejemplo Práctico (Aplicado a nuestro programa de notas)

Si aplicamos modularización al ejemplo anterior de calcular el promedio del alumno, el programa ya no sería un bloque continuo de texto, sino que se dividiría en **tres módulos independientes**:

Módulo 1: PedirNotas() -> Se encarga de solicitar los datos al usuario.

Módulo 2: CalcularPromedio(n1, n2, n3) -> Recibe las notas y devuelve el cálculo matemático.

Módulo 3: MostrarEstado(promedio) -> Evalúa si el resultado aprueba o reprueba.

Usa el código con precaución.

De esta forma, el programa principal se reduce a invocar estos módulos en orden:

1. PedirNotas()
 2. CalcularPromedio()
 3. MostrarEstado()
-

4. *Tecnologías emergentes: internet de las cosas. Aplicación a proyectos.*

El **Internet de las Cosas (IoT)** es una tecnología emergente que consiste en la interconexión digital de objetos cotidianos con Internet, permitiéndoles recopilar, enviar y recibir datos en tiempo real sin intervención humana constante. [1, 2, 3, 4]

Arquitectura de un Proyecto IoT

Cualquier proyecto de Internet de las Cosas se estructura en cuatro capas fundamentales: [1]

- **Los Dispositivos (Hardware):** Microcontroladores (como Arduino o ESP32) equipados con sensores (para medir temperatura, luz, presencia) y actuadores (motores, relés, luces). [1]

- **La Conectividad (Red):** Los protocolos físicos y de comunicación que permiten transmitir los datos recopilados (Wi-Fi, Bluetooth, Zigbee o redes móviles como 5G). [1, 2, 3, 4]

- **El Procesamiento (Nube/Servidor):** Plataformas que reciben los datos para analizarlos, almacenarlos y tomar decisiones lógicas (ej. Firebase, AWS IoT o ThingSpeak). [1, 2]

- **La Interfaz (Usuario):** Aplicaciones móviles o paneles web (*dashboards*) donde el usuario visualiza los datos en tiempo real y controla los dispositivos a distancia. [1]

Aplicación Práctica a Proyectos Reales [1]

La aplicación de IoT transforma sistemas tradicionales en entornos inteligentes mediante la automatización: [1]

- **Domótica (Casas Inteligentes):** Sistemas que regulan automáticamente la calefacción según la temperatura exterior o apagan luces si no detectan presencia en una habitación.

- **Agricultura de Precisión:** Sensores enterrados en el suelo que miden la humedad de la tierra y activan los sistemas de riego por goteo únicamente cuando la planta lo necesita.

- **Ciudades Inteligentes (Smart Cities):** Contenedores de basura que avisan al camión de recogida cuando están llenos, optimizando las rutas de limpieza urbana. [1]

- **Salud Conectada (e-Health):** Dispositivos vestibles (*wearables*) que monitorizan el ritmo cardíaco de un paciente y envían una alerta automática al médico en caso de anomalía. [1]

Conexión con los Temas Anteriores

Un proyecto IoT es la unión perfecta de todo lo que hemos visto:

1. Utiliza **programación textual o en bloques** en el microcontrolador.
2. Requiere pasar por un **proceso de desarrollo y depuración** estricto con el hardware.
3. Se diseña siguiendo la metodología de **resolución de problemas**.
4. Se estructura mediante **modularización** (un módulo lee el sensor, otro módulo se conecta al Wi-Fi y otro envía los datos)

Para diseñar el sistema de riego automatizado, crearemos una solución inteligente que mide la humedad del suelo y activa una bomba de agua solo cuando la tierra está seca. Este diseño está optimizado para simularse en **Tinkercad Circuitos**.

1. Componentes Necesarios (Hardware)

- **Cerebro:** 1 Placa Arduino Uno R3.
 - **Sensor:** 1 Sensor de humedad del suelo (mide si la tierra está seca).
 - **Actuador:** 1 Servomotor (simula la apertura de la válvula de agua) o un Relé con una Mini Bomba de agua.
 - **Indicadores:** 1 LED Verde (Riego Apagado) y 1 LED Rojo (Riego Activado).
 - **Otros:** Resistencias de $220\ \Omega$ para los LED y cables de conexión.
-

2. Conexiones en Tinkercad

• Sensor de Humedad:

- VCC a 5V de Arduino.
- GND a GND de Arduino.
- SIG (Señal) al pin analógico A0 (para leer valores variables de humedad).

• Servomotor (Válvula):

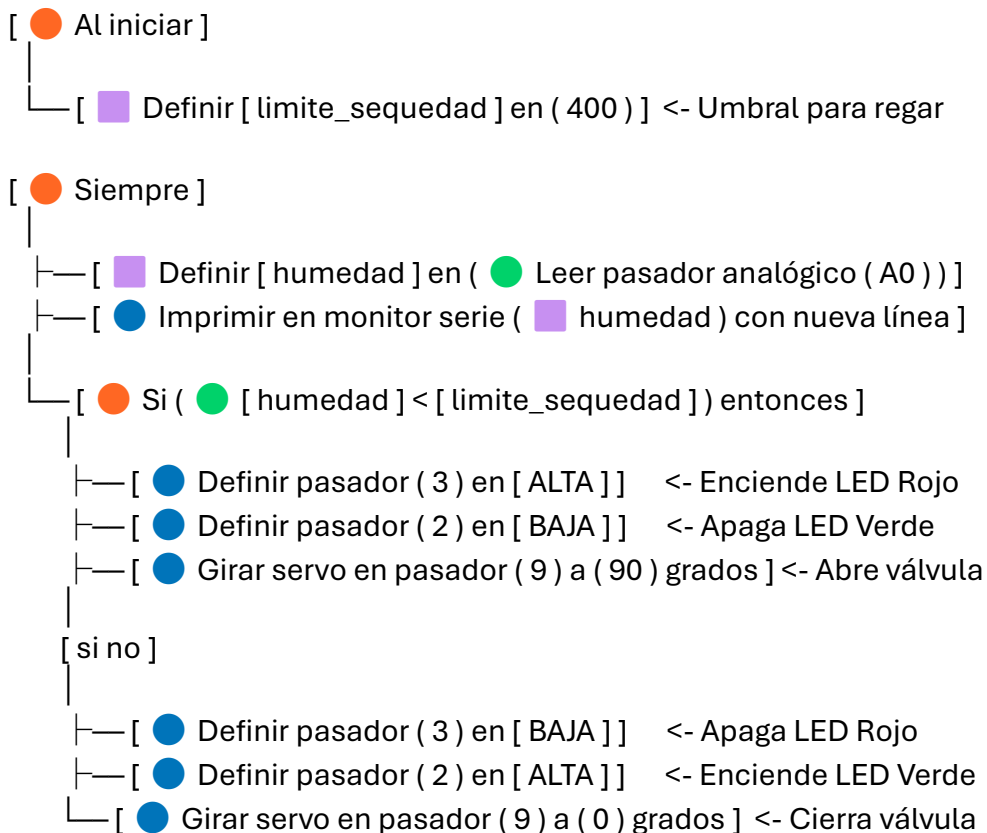
- Tierra a GND.
- Potencia a 5V.
- Señal al pin digital 9.

• LEDs:

- LED Verde al pin digital **2** (Tierra con resistencia a GND).
- LED Rojo al pin digital **3** (Tierra con resistencia a GND).

3. Programación en Bloques (Estructura Lógica)

En Tinkercad Circuitos, configuramos las variables y la condicional de la siguiente manera:



4. Flujo de Funcionamiento (El Algoritmo)

1. **Lectura:** El Arduino lee constantemente el pin **A0** para comprobar el estado de la tierra.
 2. **Decisión:** Si el valor de humedad cae por debajo de 400 (tierra seca):
 - Se enciende la luz roja de aviso.
 - El servo gira 90 grados para abrir el paso del agua.
 3. **Retorno:** Cuando el agua empapa la tierra y el sensor detecta un valor mayor o igual a 400:
 - El sistema apaga la luz roja y enciende la verde (sistema seguro).
 - El servo regresa a 0 grados, deteniendo el flujo de agua de inmediato.
-

5. *Protocolos de comunicación de redes de dispositivos*

Los protocolos de comunicación de redes de dispositivos son el **conjunto de reglas y estándares formales** que permiten que dos o más equipos de un sistema (como los componentes de nuestro sistema de riego o un servidor en la nube) se entiendan, empaqueten e intercambien información sin errores.

En el ecosistema del Internet de las Cosas (IoT) y las redes de dispositivos, estos protocolos se dividen según su nivel de operación:

1. Protocolos de Red y Enlace (Cómo se conectan físicamente)

Definen los canales de transmisión a través de los cuales viajan las ondas o pulsos eléctricos.

- **Wi-Fi (IEEE 802.11):** Alta velocidad y ancho de banda. Su consumo de energía es elevado, por lo que se usa en dispositivos conectados a la corriente eléctrica (como electrodomésticos).

- **Bluetooth / BLE (Bluetooth Low Energy):** Diseñado para transmisiones de muy corto alcance (menos de 10 metros). Consume poquísima energía, ideal para *wearables* (relojes inteligentes, sensores de salud).

- **Zigbee / Z-Wave:** Crean redes en malla (*mesh*) donde cada dispositivo repite la señal del otro. Muy usado en domótica por su bajo consumo y alta cobertura en interiores.

- **LoRaWAN:** Protocolo de largo alcance (kilómetros) y consumo ultra bajo. Transmite ráfagas de datos muy pequeñas; es el estándar para sensores agrícolas o ciudades inteligentes.

2. Protocolos de Aplicación (Cómo se envían los datos)

Definen la estructura de los mensajes de software para que la información sea procesada correctamente por las aplicaciones.

- **MQTT (Message Queuing Telemetry Transport):** Es el rey del IoT. Utiliza un modelo de **publicación/suscripción** a través de un servidor central llamado *Broker*. Es ultra ligero y perfecto para conexiones inestables o dispositivos con poca memoria.

- **HTTP / HTTPS:** El protocolo estándar de las páginas web. Utiliza peticiones *REST* (GET, POST). Aunque es muy compatible, consume mucho ancho de banda y energía para dispositivos pequeños.

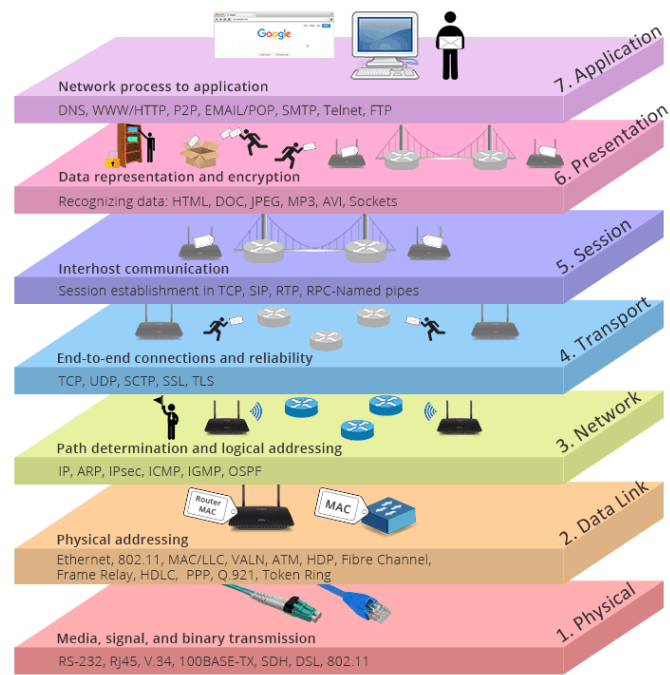
- **CoAP (Constrained Application Protocol):** Una versión ligera de HTTP adaptada para dispositivos con recursos muy limitados. Utiliza UDP para enviar datos de forma rápida sin la sobrecarga de conexiones complejas.

3. Aplicación a Nuestro Sistema de Riego

Si quisiéramos conectar el sistema de riego diseñado en el punto anterior a internet, aplicaríamos estos protocolos de la siguiente forma:

- **Capa física:** Usaríamos un microcontrolador con **Wi-Fi** (como el chip ESP32) para enviar los datos de humedad desde el jardín a la red de casa.

- **Capa de aplicación:** El dispositivo **publicaría** un mensaje mediante el protocolo **MQTT** con la lectura actual (por ejemplo: jardín/humedad = 350) hacia un servidor en la nube para que podamos ver el gráfico desde el teléfono móvil.



3. Temario automatización



1. *Sistemas de control. Conceptos y elementos. Modelización de sistemas sencillos.*

En el mundo moderno, los sistemas de control desempeñan un papel fundamental en una amplia gama de industrias y aplicaciones, desde la fabricación hasta la automatización de edificios y la robótica. Estos sistemas se utilizan para regular y supervisar procesos, asegurando que funcionen de manera eficiente, segura y confiable.

Para comprender cómo funcionan los sistemas de control, es esencial conocer los elementos básicos que los componen. En este artículo, exploraremos los componentes esenciales de un sistema de control y su importancia en la optimización de la eficiencia y el rendimiento.

- Componentes esenciales de un sistema de control:

Sensor (medición)

Un sistema de control comienza con un sensor, que es responsable de medir y recolectar datos del entorno o del proceso que se está controlando. Los sensores pueden detectar una amplia variedad de variables, como temperatura, presión, flujo, nivel y posición. La precisión y confiabilidad del sensor son cruciales, ya que cualquier error o fluctuación en las mediciones iniciales puede afectar todo el sistema de control. (En la ducha, sería tu piel sintiendo la temperatura).

Controlador (control)

El controlador es el cerebro del sistema de control. Recibe la información de los sensores, el proceso y toma decisiones para mantener el sistema en un estado deseado o responder a cambios en las condiciones del proceso. (Tu cerebro dándose cuenta de que el agua está fría).

Los controladores pueden ser de diferentes tipos, como controladores lógicos programables (PLC), controladores PID (proporcional-integral-derivativo) o controladores basados en algoritmos avanzados. El controlador ajusta las señales de salida enviadas al actuador para mantener el sistema dentro de los límites deseados y cumplir con los objetivos de control establecidos.

Actuador (acción)

El actuador es el componente encargado de realizar acciones en respuesta a las instrucciones enviadas por el controlador. Los actuadores definirán la señal de control en un cambio físico en el sistema o proceso controlado. (Tu mano girando la llave del agua caliente).

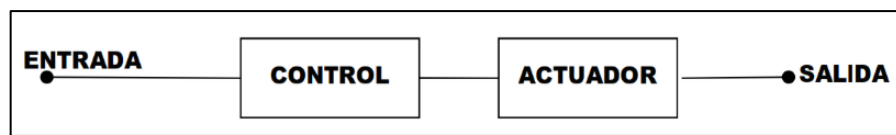
Algunos ejemplos comunes de actuadores incluyen motores, válvulas, relés y cilindros neumáticos. La elección del actuador depende de las características específicas del sistema y de los requisitos de rendimiento.

- Los dos tipos principales de sistemas de control

Existen dos formas fundamentales en las que estos sistemas operan:

1. Sistema de Lazo Abierto (Open Loop)

En este tipo, la señal de salida no tiene efecto sobre la acción de control. El sistema no mide si logró el objetivo; simplemente se ejecuta durante un tiempo o bajo una instrucción fija.

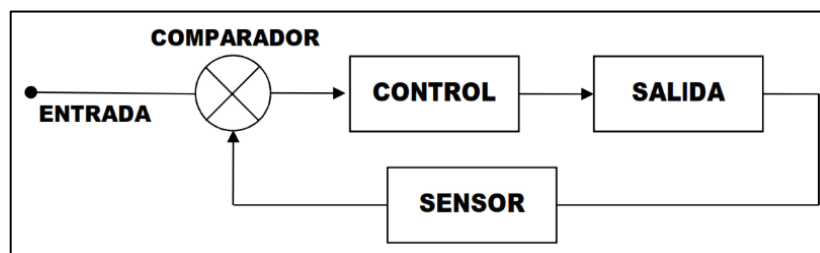


- Ejemplo clásico: Una lavadora. Tú la programas por 40 minutos y, si la ropa sale manchada, la lavadora no lo sabe ni vuelve a empezar. Solo cumple su ciclo.

- Ventaja: Son económicos y sencillos.
- Desventaja: No se adaptan a los imprevistos.

2. Sistema de Lazo Cerrado (Closed Loop o Retroalimentación)

Aquí, la salida sí se mide constantemente y se compara con el objetivo (la señal de referencia). Si hay un error, el sistema se corrige a sí mismo automáticamente. Esto se conoce como retroalimentación o *feedback*.



- Ejemplo clásico: El aire acondicionado. Si lo programas a 22°C, el sensor mide la temperatura de la habitación. Si hace calor, enciende el compresor; cuando llega a 22°C, lo apaga o reduce su potencia.

- Ventaja: Son muy precisos y capaces de compensar perturbaciones externas.

- Desventaja: Son más complejos y costosos de diseñar.

Ejemplos de la vida real

Los sistemas de control están en todas partes, desde tu casa hasta la industria espacial:

- Automatización del hogar: El control de crucero de un auto (que mantiene la velocidad constante aunque subas una colina) o los refrigeradores.
- Industria: Brazos robóticos en líneas de ensamblaje, control de presión en tuberías de gas o regulación de temperatura en reactores químicos.
- Naturaleza: Tu propio cuerpo utiliza lazo cerrado (homeostasis) para mantener la temperatura corporal a 37°C a través del sudor o los escalofríos.

2. Sistemas de supervisión (SCADA). Telemetría y monitorización. Internet de las cosas y Big Data.

Un sistema SCADA (*Supervisory Control and Data Acquisition* / Control Supervisorio y Adquisición de Datos) es el software que se coloca en la capa superior de todo el proceso que hemos estado modelando.

Si el sensor es el "ojo", el motor es el "músculo" y el PLC (Controlador Lógico Programable) es el "cerebro local", el SCADA es el "centro de comando" desde donde los operadores humanos vigilan y dirigen toda la planta.



- ¿Qué hace exactamente un SCADA?

Un sistema SCADA cumple con cuatro funciones críticas en tiempo real:

1. **Supervisión:** Muestra de forma gráfica y amigable (pantallas HMI con dibujos de tanques, tuberías, motores) el estado actual de la planta.
2. **Adquisición de Datos:** Recopila las lecturas de los sensores, variables de proceso, estados de válvulas, etc., y las centraliza.
3. **Control:** Permite al operador enviar comandos manuales (ej. "abrir válvula X", "cambiar Setpoint a 50°C") que bajan hacia los controladores físicos.

4. **Almacenamiento y Alarmas:** Guarda un registro histórico de datos (para auditorías o gráficas de rendimiento) y avisa visual o acústicamente si una variable (como la presión de un tanque) sale de los límites seguros.

- **Arquitectura de un Sistema SCADA Sencillo**

Para que los datos pasen desde el proceso físico hasta la pantalla de la computadora, se necesita una estructura en niveles:

1. Nivel de Campo (Los elementos físicos)

Son los componentes que interactúan directamente con el proceso.

- **Sensores:** Miden temperatura, flujo, presión, etc.
- **Actuadores:** Motores, válvulas, pistones, calentadores.

2. Nivel de Control (La inteligencia local)

Son los dispositivos electrónicos que ejecutan los bucles de control (como los modelos PID que vimos antes).

- **PLC (Controlador Lógico Programable):** Computadoras industriales robustas que reciben las señales de campo y controlan el proceso en microsegundos.

- **RTU (Unidad Terminal Remota):** Dispositivos similares al PLC pero optimizados para enviar datos a larguísimas distancias (muy usados en oleoductos o redes eléctricas).

3. Nivel de Supervisión (El Software SCADA)

Es la computadora (o servidor) que corre el software SCADA (ejemplos comerciales: *Ignition*, *Wonderware InTouch*, *WinCC*, *Siemens*).

- Se comunica con los PLCs a través de redes y protocolos industriales (como Modbus, Profinet o el estándar moderno **OPC UA**).

- Traduce los miles de datos numéricos que envían los PLCs en animaciones gráficas para el operario.

- **Un ejemplo práctico: SCADA en una Planta Potabilizadora de Agua**

Imagina el sistema de control de nivel de agua que modelamos anteriormente. En una planta real, no hay solo un tanque, hay decenas de ellos repartidos en kilómetros.

- **Sin SCADA:** Un operario tendría que caminar bajo la lluvia hasta el tanque 5 para ver un manómetro, apuntar el nivel en una libreta y girar una manivela para cerrar la entrada de agua si se está desbordando.

- **Con SCADA:** El operario está sentado en una sala con aire acondicionado frente a tres monitores. La pantalla muestra el Tanque 5 parpadeando en **rojo** porque el nivel superó el 90% (Alarma). Desde su teclado, el operario hace clic en la válvula de entrada, cambia su estado a "Cerrada", y el SCADA envía la orden al PLC de campo para que corte el flujo. Todo en menos de un segundo.

- Diferencia clave: SCADA vs. HMI

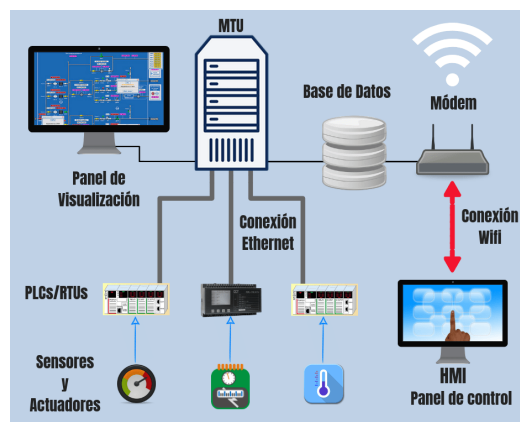
Es común confundirlos porque ambos tienen pantallas gráficas:

- **HMI (Human-Machine Interface):** Es una pantalla táctil pequeña pegada físicamente al lado de una máquina específica (ej. la pantallita de un horno industrial). Solo controla esa máquina.

- **SCADA:** Es un sistema de software global, centralizado en una sala de control, que supervisa e integra múltiples HMIs, PLCs y procesos de toda una fábrica o incluso de una región geográfica entera.

Para cerrar el círculo de la automatización, entramos en la parte que hace posible que el SCADA reciba la información: **la telemetría y la monitorización**.

Aunque suenen a palabras espaciales o de Fórmula 1, son conceptos muy sencillos que usamos todos los días.



• ¿Qué es la Telemetría? (Medir a distancia)

La palabra viene del griego *tele* (lejos) y *metron* (medida). Es la tecnología que permite **medir una variable física en un lugar y enviar ese dato automáticamente a otro lugar lejano** para que alguien lo analice.

El ejemplo de la Fórmula 1

Es el caso más famoso. Mientras Lewis Hamilton va a 300 km/h en la pista, no puede mirar la pantalla para ver el desgaste de sus neumáticos o la temperatura del motor.

- Los **sensores** del coche miden esos datos.
- Una **antena** los envía por radio en milisegundos.
- Los ingenieros en el box (a cientos de metros) ven en sus pantallas que el coche se está quedando sin frenos. **Eso es telemetría.**

• ¿Qué es la Monitorización? (Vigilar en tiempo real)

Si la telemetría es la que *lleva* los datos de un lado a otro, la monitorización es **la acción de observar y analizar esos datos de forma continua para ver si todo funciona bien**. Monitorizar es "estar vigilando".

El ejemplo del hospital

Cuando una persona está ingresada y le ponen pinzas en los dedos y chupones en el pecho conectados a una pantalla que hace *beep... beep... beep...*

- La pantalla está **monitorizando** sus constantes vitales (pulsaciones, oxígeno).
- Si el corazón late demasiado rápido, la pantalla pita (salta una alarma).

La diferencia y cómo se unen

A veces se confunden, pero la relación es muy clara: **Usas la telemetría para hacer monitorización a distancia.**

Concepto	¿Qué hace?	Ejemplo en la industria
Telemetría	Envía el dato.	Un sensor de boya en un embudo en mitad de la montaña envía por satélite el nivel del agua.
Monitorización	Muestra y vigila el dato.	El operario en la ciudad ve en una pantalla una gráfica del nivel de ese embudo para vigilar que no haya sequía.

Si unimos lo que ya hemos visto (sensores, telemetría y SCADA) con internet y ordenadores gigantescos, nacen el **Internet de las Cosas (IoT)** y el **Big Data**. En el

mundo del control, esto es lo que permite pasar de controlar una sola fábrica a controlar "el mundo entero".

• Internet de las Cosas (IoT / Internet of Things)

El IoT consiste, simplemente, en **conectar a internet objetos cotidianos o máquinas industriales que antes no estaban conectados**, para que puedan hablar entre ellos y con nosotros.

¿Cómo funciona?

En lugar de tener un sensor conectado por un cable a un PLC (como en el control clásico), en el IoT el propio sensor tiene un chip Wi-Fi o 5G. El sensor mide el dato y lo sube directamente a "la nube" (a internet).

En la industria (IIoT):

Imagina que una empresa de paquetería tiene 500 camiones repartidos por toda Europa. Con IoT, cada camión lleva sensores en el motor, en las ruedas y en la nevera de la carga. Todos esos datos se envían por internet en tiempo real a la central.



• Big Data (El océano de datos)

¿Qué pasa cuando tienes 500 camiones enviando datos cada segundo, o una fábrica con 10.000 sensores IoT funcionando las 24 horas? Que te inundas de información. Un ordenador normal o un SCADA clásico colapsarían.

Ahí entra el **Big Data**: es el conjunto de tecnologías y superordenadores capaces de **almacenar, ordenar y analizar cantidades gigantescas de datos** que se generan a toda velocidad.

- **¿Para qué sirve en los sistemas de control? (El "Superpoder" Predictivo)**

El Big Data no solo mira lo que pasa *ahora* (eso lo hace el SCADA). El Big Data busca patrones en el histórico de millones de datos para **predecir el futuro**.

• **Mantenimiento predictivo:** El sistema analiza las vibraciones de una pieza de un tren durante meses. De repente, detecta una vibración microscópica idéntica a la que provocó una avería hace tres años. El sistema avisa: *"Cambia esa pieza la semana que viene o se romperá en mitad del trayecto"*.



3. Robótica: modelización de movimientos y acciones mecánicas.

Si un robot fuera un ser humano, el **SCADA** y el **IoT** serían su conexión con el mundo, pero la **modelización de movimientos** es la forma en que su cerebro le dice a sus músculos (motores) cómo moverse sin chocar, romper cosas o caerse.

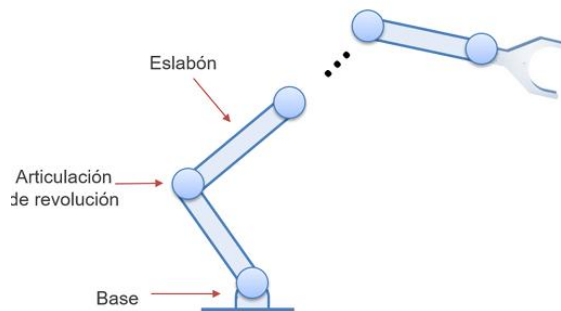
Aquí tienes cómo explicarles la física y las matemáticas que hacen que un robot se mueva con precisión milimétrica.

1. El "Esqueleto" del Robot: Eslabones y Articulaciones:

Para que el cerebro del robot sepa dónde está su mano (llamada **efector final**), primero debemos modelar cómo está construido. Un robot se modela igual que el cuerpo humano:

• **Eslabones (Links):** Son las partes rígidas (como tus huesos).

• **Articulaciones (Joints):** Son los puntos de unión que permiten el movimiento (como tus codos o rodillas). Pueden ser **rotacionales** (giran) o **prismáticas** (se estiran como un telescopio).



2. Modelización del Movimiento: ¿Hacia dónde me muevo?:

En robótica hay dos formas matemáticas de calcular el movimiento. Para explicárselo a los alumnos, imagina que el robot quiere coger una manzana:

A. Cinemática Directa (El camino fácil)

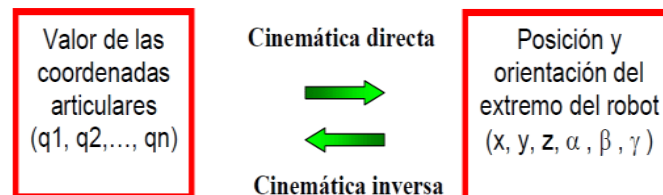
Tú le dices al robot cuánto tiene que girar cada uno de sus motores.

- **La pregunta matemática:** "Si el motor del hombro gira 45° y el del codo gira 90° , ¿en qué coordenadas (x, y, z) del espacio se ha quedado la mano?"
- **Para clase:** Es como cuando juegas con un muñeco articulado y le mueves el brazo manualmente.

B. Cinemática Inversa (El verdadero reto)

Aquí es al revés. Tú solo le dices al robot la coordenada exacta en la que está la manzana. El ordenador del robot tiene que calcular (usando trigonometría avanzada) cuánto debe girar cada motor para llegar ahí.

- **La pregunta matemática:** "La manzana está en la posición $x=5, y=2, z=3$. ¿Cuánto tienen que girar mis motores para que la mano llegue exactamente ahí sin pasarse?"
- **Para clase:** Es el cálculo mental automático que hace tu cerebro cuando alargas la mano para coger el móvil de la mesa.



3. Acciones Mecánicas: Fuerzas y Dinámica:

Moverse en el aire es fácil, pero un robot real interactúa con el mundo físico. La **dinámica** modela las fuerzas implicadas:

- **Inercia y Peso:** El robot tiene que saber cuánto pesa su propio brazo y el objeto que va a levantar. No se necesita la misma fuerza en el motor para levantar una pluma que una caja de acero.
- **Actuadores (Los músculos):** Normalmente son **servomotores**. A diferencia de un motor normal, a un servomotor le puedes decir: "*Gira exactamente 32 grados y quédate clavado ahí*".

4. Automatización programada de procesos. Diseño, programación, construcción y simulación o montaje.

La **automatización programada** es un nivel intermedio dentro de la automatización industrial. A diferencia de la automatización fija (donde las máquinas hacen siempre la misma tarea mediante mecanismos rígidos), la automatización programada permite **reconfigurar y adaptar el sistema mediante software** para producir diferentes tipos de productos o cambiar las secuencias del proceso.

Es el núcleo de la producción por lotes (*batch production*), donde las máquinas se reprograman cada vez que termina una campaña de producción y empieza una nueva.

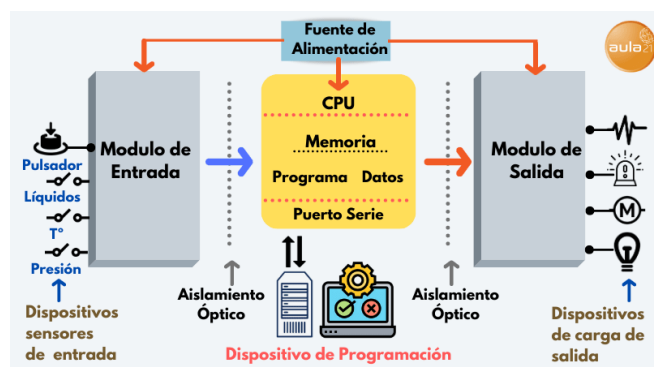
1. El Núcleo Tecnológico: El Autómata Programable (PLC)

El cerebro indiscutible de la automatización programada es el **PLC (Programmable Logic Controller)**. Es un ordenador de diseño industrial, altamente robusto frente al ruido eléctrico, polvo, vibraciones y temperatura, que ejecuta un programa de forma cíclica. Es el sistema que hace que la maquinaria y los sistemas funcionen automáticamente.

Ciclo de funcionamiento de un PLC:

El PLC funciona en un bucle cerrado infinito que consta de tres pasos principales:

1. **Lectura de Entradas:** Lee el estado de los sensores (sensores ópticos, finales de carrera, termopares, etc.).
2. **Ejecución del Programa:** Procesa la lógica programada en su memoria basándose en el estado de esas entradas.
3. **Escritura de Salidas:** Activa o desactiva los actuadores (motores, electroválvulas, relés, variadores de frecuencia).



2. Lenguajes de Programación Estándar (IEC 61131-3)

Para programar estos procesos, la norma internacional **IEC 61131-3** define los lenguajes estándar que un estudiante de Bachillerato o Ingeniería debe conocer. Se dividen en dos grandes grupos:

Lenguajes Gráficos

- **LD / Ladder (Diagrama de Contactos):** Basado en los antiguos esquemas de relés eléctricos. Utiliza "contactos" (entradas) y "bobinas" (salidas) dispuestos en líneas horizontales como si fueran los peldaños de una escalera. Es el más utilizado a nivel mundial por su facilidad de diagnóstico.

- **FBD (Diagrama de Bloques de Funciones):** Representa las variables y funciones como bloques con entradas a la izquierda y salidas a la derecha. Muy intuitivo si se tienen nociones de puertas lógicas (*AND*, *OR*, *NOT*).

- **SFC / Grafcet (Gráfico de Orden de Etapas y Transiciones):** Ideal para modelar procesos secuenciales. Divide el proceso en **Etapas** (acciones que ocurren en un momento dado) y **Transiciones** (condiciones lógicas que deben cumplirse para pasar a la siguiente etapa).

Lenguajes Textuales

- **ST (Texto Estructurado):** Muy similar a lenguajes de programación de alto nivel como *C* o *Pascal*. Utiliza estructuras condicionales (*IF...THEN*, *CASE*) y bucles (*FOR*, *WHILE*). Se utiliza para cálculos matemáticos complejos dentro del control de procesos.

3. Ejemplo Práctico: Modelado de un Proceso de Mezcla Secuencial

Para ilustrar cómo se programa un proceso automatizado, analicemos una secuencia básica en una planta de bebidas mediante la lógica de etapas y transiciones (Grafcet):

- **Etapas 0 (Reposo):** El sistema espera a que el operador pulse el botón de inicio (Start).

- *Transición:* Start activado.

- **Etapas 1 (Llenado):** Se abre la electroválvula A ($VA = 1$) para llenar un tanque con líquido base.

- *Transición:* El sensor de nivel alto (S_{Alto}) se activa.

- **Etapas 2 (Mezcla):** Se cierra VA y se activa el motor del agitador ($Motor = 1$) durante un tiempo de 30 segundos.

- **Transición:** Temporizador finalizado ($T1 = 30s$).

• **Etapa 3 (Vaciado):** Se apaga el motor y se abre la válvula de evacuación B ($VB = 1$) hacia la embotelladora.

• **Transición:** El sensor de nivel bajo (S_Bajo) detecta que está vacío. El sistema regresa a la **Etapa 0**.

4. Ventajas y Limitaciones de este Tipo de Automatización

Ventajas:

• **Flexibilidad:** Modificar el proceso o el producto no requiere cambiar el cableado físico ni las máquinas; basta con modificar las líneas de código en el PLC.

• **Escalabilidad:** Permite integrar funciones de seguridad industrial, contadores, temporizadores y comunicación con sistemas superiores (como el SCADA que vimos anteriormente).

Limitaciones:

• **Tiempo de reconfiguración:** Aunque es programable, cambiar de un lote de producción a otro requiere detener la línea, cargar el nuevo programa y, a menudo, hacer ajustes mecánicos manuales. *(Para evitar esto, la industria actual evoluciona hacia la Automatización Flexible).*

- El Ciclo de Vida de un Proyecto Tecnológico: Del Papel a la Realidad

Cuando queremos automatizar un proceso (ya sea una maqueta de un invernadero inteligente en clase o una línea de producción real), no se empieza directamente a conectar cables y programar. Se sigue un método de ingeniería dividido en cuatro grandes fases secuenciales.

1. DISEÑO: Planificar antes de actuar

Esta es la fase analítica. El objetivo es dejar claro qué va a hacer el sistema y qué elementos necesitamos para construirlo, evitando comprar componentes a ciegas.

• **El "Plano" del Proceso (Diagrama de Flujo / Grafcet):** Antes de programar, los alumnos deben dibujar el comportamiento de la máquina. Si es un invernadero, el flujo será: *Si la humedad baja del 40% → activar bomba de agua → esperar hasta llegar al 60% → apagar bomba.*

• **El Diseño Eléctrico:** Se seleccionan los sensores y actuadores compatibles con el cerebro del sistema (por ejemplo, una placa Arduino o un mini-PLC). Se

dibuja un esquema de conexiones para saber exactamente a qué pin o borna irá cada cable.

- **El Diseño Mecánico:** Si el proyecto requiere piezas físicas (un brazo que se mueve, un chasis), se diseñan en ordenador usando programas de modelado **CAD 3D** (como *Tinkercad* para iniciación o *Onshape* / *Fusion 360* en niveles avanzados).

2. PROGRAMACIÓN: Dar órdenes al sistema

Con los planos eléctricos terminados, se pasa a escribir el software. En Bachillerato, dependiendo de la tecnología elegida, se trabaja con dos entornos:

- **Programación por bloques o texto:** Se pasa la lógica que diseñamos en la fase anterior al código definitivo (usando lenguajes como C++ para Arduino, *Python* o diagramas de contactos *Ladder* si se usan micro-PLCs industriales).

- **Gestión de variables:** Los alumnos aprenden a mapear los pines. El pin digital 2 ya no es solo un número; en el programa se define como la variable `SENSOR_PRESENCIA`, y el pin 3 como `MOTOR_CINTA`.

- **Creación de la Interfaz (HMI básica):** Se programa cómo verá el usuario los datos (por ejemplo, mostrando la temperatura actual en una pequeña pantalla LCD o en una aplicación móvil conectada por Bluetooth).

3. SIMULACIÓN Y CONSTRUCCIÓN: Validación virtual y montaje del hardware

Para evitar cortocircuitos o romper componentes caros, hoy en día la simulación es un paso obligatorio antes de tocar las piezas reales.

- **Simulación Virtual (El gemelo digital):** Antes de armar el circuito real, se testea el programa en simuladores interactivos en línea (como *Tinkercad Circuits* o *Wokwi*). Aquí los alumnos cargan su código y ven si el motor virtual gira cuando el sensor virtual detecta luz. Si hay un error de código, el circuito virtual "falla" pero ningún componente real sufre daños.

- **Construcción del Hardware:** Una vez validado en el simulador, se pasa al taller. Incluye la fabricación de piezas (mediante **impresión 3D** o corte de madera/metacrilato) y el cableado físico de los componentes en una placa de pruebas (*protoboard*) o soldando los cables definitivos.

4. MONTAJE Y PUESTA EN MARCHA: El test definitivo en el mundo real

Es la fase final del proyecto, donde el software y el hardware se unen en su entorno definitivo.

- **Integración:** Se montan los circuitos dentro de la estructura mecánica de la maqueta o prototipo.

- **Prueba de Entradas y Salidas (I/O Checking):** Antes de lanzar el programa completo, se hace un testeo manual. Se comprueba con un polímetro si llega corriente a los motores y si los sensores cambian de estado correctamente al pasar la mano frente a ellos.

- **Puesta en marcha y Calibración:** Se carga el programa final y se observa el comportamiento real. Aquí los alumnos se enfrentan a los "problemas del mundo real" que la simulación no contempla: rozamientos mecánicos, interferencias en los cables o la necesidad de calibrar un sensor porque la luz ambiental de la clase no es la misma que la de su casa.

5. *Aplicación de las tecnologías emergentes a los sistemas de control.*

Las tecnologías emergentes no destruyen el control clásico (los PLCs o los lazos cerrados que ya hemos estudiado), sino que se colocan "por encima" de ellos para dotarlos de mayor inteligencia, flexibilidad y capacidad de comunicación.

1. Gemelos Digitales (Digital Twins)

Un gemelo digital es una **réplica virtual exacta, en tiempo real, de un sistema físico** (una máquina, un brazo robótico o una planta entera).

- **¿Cómo se aplica al control?** El gemelo digital recibe constantemente los datos de la telemetría y los sensores del sistema real. Si el PLC de la planta cambia un parámetro, el gemelo digital predice inmediatamente el impacto.

- **Aplicación práctica:** Permite probar cambios de programación agresivos o simular catástrofes en el modelo virtual para ver cómo reaccionaría el sistema de control real, sin poner en riesgo la fábrica.

2. Inteligencia Artificial (IA) y Machine Learning

El control clásico se basa en algoritmos matemáticos rígidos (como el PID, que responde siempre de la misma manera ante el mismo error). La IA introduce el **control adaptativo e inteligente**.

- **Optimización en tiempo real:** Los algoritmos de *Machine Learning* analizan millones de datos históricos de la planta y pueden reconfigurar automáticamente los parámetros del controlador para ahorrar energía o reducir el desgaste de las piezas según las condiciones ambientales del día.

- **Mantenimiento Predictivo:** En lugar de esperar a que una máquina falle (control reactivo) o cambiar piezas por calendario (control preventivo), la IA detecta

anomalías microscópicas en el consumo eléctrico o las vibraciones de un motor y avisa con semanas de antelación de que una pieza va a fallar.

3. Edge Computing (Computación en el Borde)

Tradicionalmente, en los sistemas IoT, los sensores envían los datos a un servidor central en "la nube" (servidores en internet) para ser analizados. Sin embargo, en los sistemas de control, **el tiempo de respuesta debe ser de milisegundos**. Si un robot va a chocar, no puede esperar a que los datos viajen a un servidor en EE.UU. y vuelvan.

- **¿Qué es?** Consiste en colocar la capacidad de procesamiento físico y los algoritmos de IA **en el propio dispositivo de campo** (el PLC o un ordenador industrial pegado a la máquina).

- **Aplicación práctica:** El procesamiento de datos masivos se hace de forma local e instantánea (*Edge*), y solo los datos útiles o los resúmenes históricos se suben a la nube.

4. Realidad Aumentada (RA) en la Supervisión

La evolución de los sistemas SCADA tradicionales hacia las tecnologías inmersivas.

- **¿Cómo se aplica?** Los operarios o técnicos de mantenimiento utilizan gafas de Realidad Aumentada (como las *HoloLens*) o tablets. Al enfocar con la cámara a un cuadro eléctrico o a un motor físico, el sistema superpone gráficos virtuales en tiempo real sobre la imagen real.

- **Aplicación práctica:** El técnico puede ver la temperatura interna del motor, el voltaje exacto que está pasando por un cable o los pasos detallados para reparar una avería flotando en el aire justo al lado de la máquina.

- Resumen de Integración Tecnológica -

Para que los alumnos asimilen el esquema completo, podemos resumir cómo interactúan todas las tecnologías que hemos visto a lo largo del temario:

1. El **Sensor** mide el mundo físico.
2. El **PLC/Controlador** ejecuta el lazo cerrado local en milisegundos.
3. El **Edge Computing** filtra y procesa esos datos al instante utilizando **IA** local.
4. El **SCADA** centraliza la información para el ojo humano.
5. El **IoT** sube los datos globales a la nube para alimentar al **Gemelo Digital**.
6. El **Big Data** analiza el histórico para predecir el futuro de toda la infraestructura.



IES GREGORI MAÏANS
OLIVA